

2) switch to another branch and delete test-branch:
git checkout main

Date ___/___/___
Page ___

6a

git log -n 5 --online
git revert <commit-ids>

6b

git config --global alias.st status
git config --global alias.lg "log --online"

~~6a~~ git config --global --list

git config --global --list | grep alias

7a

git revert -m 1 <commit-ids>

7b

git branch -D feature

↳ This will give an id

git checkout -b feature <id>

8a

git clean -n # To preview

git clean -f # For untracked files

git clean -fd # For untracked dir.

★ 8b

git archive --format=zip --output=main.branch.zip main

git archive --format=tar --output=main.tar main

1A:

```
git init
git touch a.txt
touch b.txt
git add .
git commit -m "first commit"
```

1B

```
touch a.txt
git add .
git commit
git checkout -b feature
# To bring conflict, change content in master
branch and then commit it.
Then switch to main feature branch, do
changes & then merge from master
gedit a.txt → add & commit
git checkout master
git merge feature
```

★ 2A.

```
git touch a.c
git add & commit
git checkout -b feature
touch b.c
git add .
git add stash
git checkout master
git stash apply
```

2B. git clone "1" →

- ① git stash - all in every file
untrack, track, & ignore
- ② git stash create
- ③ git stash pop → applies & del
top stash
- ④ git stash drop stash @1 of
(most recent or specific del)
- ⑤ git stash clear
(del everything)
- ⑥ git stash list

★ 3A. rebase

git clone

→ add new file or do changes in remote repo

→ Do some changes locally (should not conflict with remote changes)

→ git fetch

→ git rebase origin/main

3B

git merge feature -m "message"

4A

git tag v1 <commit-id>

★ 4B cherry-pick

git checkout master

git checkout a.c

git add & commit

git checkout -b "feature"

touch b.c

git add & commit X3

git checkout master

git cherry-pick <oldest> 1..<newest>

★ 5A

git show <commit-id>

★ 5B

git log --author="aniket" since="2026-2-23" --until="2025-03-23"